OFFICIAL TECHNICAL DOCUMENTATION • ENTERPRISE EDITION

CareFlow — Technical Project Manual

Smart Hospital OPD & Live Queue Management System • Android Architecture Specification

AUTHOR & LEAD DEVELOPER
Labhesh Wayakole

TARGET PLATFORM
Android SDK 36 (Android 14+)

CLOUD INFRASTRUCTURE
Google Cloud Firestore & Auth

PACKAGE NAMESPACE
`com.example.careflow`

1. Executive Summary & Healthcare Problem Definition

In modern multi-specialty hospitals and medical outpatient departments (OPDs), physical waiting lines and chaotic waiting rooms cause significant patient distress, unmonitored infection risks, and physician schedule unpredictability. Patients often wait 2 to 4 hours with no visibility into their consultation sequence, leading to crowded lobbies and high missed-turn rates.

CareFlow is an enterprise-grade native Android healthcare application engineered to transform outpatient hospital throughput. It replaces physical queues with **Real-Time Digital Token Passes**, **Dynamic Wait-Time Estimation Telemetry**, **One-Tap Doctor Summoning with High-Urgency Audio-Visual Call Alerts**, and a centralized **Hospital Command Center** for multi-department operations.

2. Portal Architecture & System Capabilities

PORTAL	PRIMARY USERS	KEY ARCHITECTURAL CAPABILITIES
Patient Portal	Patients, Family Attendants	Hospital selection (CareFlow, Apollo, Max, Fortis, Manipal), specialty department discovery, doctor profiles with live OPD presence, 1-tap zero-friction token generation, live queue tracker with dynamic countdown, full-screen audio-visual call alerts.
Doctor Portal	Physicians, Specialists	Chamber queue director, active token counter, patient consultation status transitions (`PENDING` ➔ `CALLED` ➔ `IN_CONSULTATION` ➔ `COMPLETED` / `SKIPPED`), digital clinical notes and prescription generator, presence and break-mode toggle.
Admin Portal	Medical Directors, Administrators	Hospital profile and OPD concourse configuration, staff onboarding with auto-generated secure doctor credentials (Doctor ID, email, password), department architecture manager, doctor password reset approval, live OPD telemetry.

3. Technical Stack & Clean Architecture Specification

LAYER	TECHNOLOGY	ARCHITECTURAL RESPONSIBILITY & JUSTIFICATION
UI Toolkit	Jetpack Compose (Material 3)	100% declarative UI with zero XML layouts; state hoisting; accessible contrast palettes; GPU-accelerated compositing.
Asynchronous Core	Kotlin Coroutines & Flow	Non-blocking reactive data streams (`StateFlow`, `callbackFlow`); background I/O execution on `Dispatchers.IO`.
Architecture	Clean Architecture + MVVM	Unidirectional Data Flow (UDF); strict separation between Presentation, Domain, and Data tiers; `ServiceLocator` DI.
Cloud Database	Cloud Firestore NoSQL	Real-time bi-directional WebSocket streams (`addSnapshotListener`); sub-50ms synchronization across patient and doctor screens.
Cloud Auth	Firebase Authentication	Email/Password, Google 1-Tap Sign-In, Phone OTP verification with secure role binding and token lifecycle.
Local Persistence	Encrypted SharedPreferences	Role-aware session persistence (`UserSessionManager`) with multi-role state isolation and instant app boot.
Resilience Engine	Dual-Engine InMemory Store	High-fidelity local fallback data store (`CareFlowSeedData`) ensuring 100% offline functionality without blank screens.

4. Cloud Firestore Database Schema

1. `/hospitals/{hospitalId}` (Healthcare Institutions)
Stores hospital master records: `id`, `name`, `opdName`, `address`, `city`, `phone`, `email`, `adminId`, `adminEmail` (single-admin authority), `adminName`, `adminPhone`, `departmentIds[]`, `createdAt`, `updatedAt`.

2. `/doctors/{doctorId}` (Physicians & Specialists)
Stores credentialed clinicians: `id`, `userId`, `name`, `specialization`, `departmentId`, `departmentName`, `hospitalId`, `qualification`, `experienceYears`, `consultationFee`, `avgConsultationMinutes`, `shiftStartTime`, `shiftEndTime`, `maxTokensPerDay`, `isAvailable`, `isInOPD`, `loginEmail`, `password`.

3. `/appointments/{appointmentId}` (OPD Consultation Bookings)
Stores live consultation passes: `id`, `patientId`, `patientName`, `patientPhone`, `doctorId`, `doctorName`, `departmentId`, `departmentName`, `hospitalId`, `tokenNumber`, `date`, `timeSlot`, `status` (`PENDING`, `CALLED`, `IN_CONSULTATION`, `COMPLETED`, `CANCELLED`, `ABSENT`), `notes`, `symptoms`, `bookedAt`.

4. `/queues/{doctorId_date}` (Daily Live Queue Telemetry)
Stores active queue state: `doctorId`, `date`, `currentToken` (currently serving), `totalTokens`, `status` (`WAITING`, `ACTIVE`, `PAUSED`, `CLOSED`), `breakMessage`, `updatedAt`.

5. `/users/{userId}` (User Profiles & Role Binding)
Stores user accounts: `id`, `name`, `email`, `phone`, `role` (`PATIENT`, `DOCTOR`, `ADMIN`), `hospitalId`, `selectedHospitalId`, `fcmToken`, `createdAt`.

6. `/doctor_reset_requests/{requestId}` (Doctor Security Reset Requests)
Stores clinician self-service password reset requests: `id`, `doctorId`, `doctorName`, `doctorEmail`, `hospitalId`, `otp`, `status` (`PENDING`, `VERIFIED`, `DISMISSED`), `requestedAt`, `expiresAt`.

5. Multi-Role Profile Isolation & Single-Admin Invalidation Architecture

CareFlow incorporates two enterprise-grade security protocols addressing complex healthcare access patterns:

Protocol A: Multi-Role Account Isolation Under One Email
A healthcare worker can simultaneously be a hospital administrator, a consulting doctor, and a patient under the exact same email address (e.g. `doctor@hospital.com`). CareFlow guarantees strict isolation by qualifying accounts via composite tuple keys `(email, portalRole)`. Logging into the Patient Portal creates/binds a dedicated Patient profile (`patient_...`) with appointments and medical history; logging into Doctor Portal binds the clinician chamber; logging into Admin Portal verifies administrative authority. Sessions and data are completely segregated with zero cross-contamination.

Protocol B: Strict Single-Admin Invalidation Engine
To prevent orphaned administrative access, each hospital designates exactly one authoritative `adminEmail` on the hospital record. When the administrator updates their email in Admin Settings: (1) `hospitals/{id}.adminEmail` is atomically updated, (2) all previous admin user documents in Cloud Firestore are immediately downgraded from `ADMIN` to `PATIENT`, and (3) any login attempt with the previous email into the Admin Portal is immediately rejected with an explicit access-denied error.

6. Complete Project Directory Structure

```
CareFlow/
├── app/
│   ├── src/main/java/com/example/careflow/
│   │   ├── CareFlowApp.kt              # Application entry & notification channel setup
│   │   ├── MainActivity.kt             # Single-Activity Jetpack Compose host
│   │   ├── Navigation.kt               # Navigation 3 architecture & role-based routing
│   │   ├── core/                       # Cross-cutting system modules
│   │   │   ├── di/ServiceLocator.kt    # Singleton dependency container
│   │   │   ├── dispatcher/             # Coroutine Dispatcher providers
│   │   │   ├── notification/           # CallNotificationService & Audio-Visual overlay
│   │   │   └── result/Resource.kt      # Sealed Resource<T> (Success, Error, Loading)
│   │   ├── domain/                     # Pure business logic & interfaces (zero Android SDK)
│   │   │   ├── model/                  # Hospital, Doctor, Appointment, DailyQueue, User
│   │   │   ├── repository/             # AuthRepository, AppointmentRepository, DoctorRepo...
│   │   │   ├── usecase/                # BookAppointment, AdvanceQueue, CalculateWaitTime...
│   │   │   └── util/                   # BookingTimeUtils (time slot generators)
│   │   ├── data/                       # Concrete implementations & data sources
│   │   │   ├── datasource/             # InMemoryDataStore & CareFlowSeedData fallback
│   │   │   ├── firebase/               # FirebaseDataSource (Auth & Firestore integration)
│   │   │   ├── local/                  # UserSessionManager (SharedPreferences cache)
│   │   │   └── repository/             # Repository implementations
│   │   ├── presentation/               # Material 3 UI screens & ViewModels
│   │   │   ├── admin/                  # AdminDashboardScreen & AdminViewModel
│   │   │   ├── auth/                   # Multi-portal AuthScreen & AuthViewModel
│   │   │   ├── doctor/                 # DoctorDashboardScreen & DoctorViewModel
│   │   │   ├── patient/                # DoctorBookingScreen, QueueTrackerScreen, Overlays
│   │   │   └── components/             # Common badges, status cards, action buttons
│   │   └── theme/                      # Color palettes, Typography, Theme definition
│   ├── res/raw/queue_call_alert.wav    # High-urgency audio chime for doctor calls
│   └── build.gradle.kts                # Android SDK 36, Compose BOM, Firebase dependencies
├── firestore.rules                     # Declarative Cloud Firestore Security Rules v2
├── README.md                           # Comprehensive GitHub documentation
└── CareFlow_Project_Manual.pdf         # Official Technical Project Manual (This Document)
```

7. Operations, Build & Testing Verification

1. Compiling and Packaging the Production APK:

```
# Check Kotlin compilation across all layers
./gradlew compileDebugKotlin

# Assemble the complete debug APK
./gradlew assembleDebug

# Output APK path:
app/build/outputs/apk/debug/app-debug.apk
```

2. Automated Architecture & Unit Verification:

```
# Run domain, usecase, and data repository unit test suite
./gradlew testDebugUnitTest

# Validate Cloud Firestore security rules locally via Firebase CLI Emulator
firebase emulators:start --only firestore
```